

PROJET E-COMMERCE

Resume architecture, objectifs et feuille de route de developpement

Plateforme e-commerce industrielle, OSS-first, multi-site, event-driven, securisee, observable et reproductible.

1. Objectif du projet

Construire une plateforme e-commerce complete, performante, securisee, hautement disponible et industrialisee, capable de fonctionner sur deux sites actifs, avec une strategie initiale 100 % open source / self-hosted autant que raisonnablement possible.

Le produit doit etre developpe et opere par les equipes internes Dev, DevOps, SRE, Platform Engineering et Data, tout en restant portable vers des services manages si le contexte economique evolue ou si le site est vendu.

2. Cible metier

Le modele retenu est un retailer mono-vendeur, et non une marketplace multi-vendeurs.

- Decouverte produit et catalogue
- Recherche et discovery
- Pricing et promotions
- Inventory / stock
- Cart et checkout
- Tax
- Fraud/Risk
- Payment
- Order
- Shipping et Tracking
- Billing / facturation
- Notification
- Returns / Refund
- Reviews

Toute decision metier, securite ou etat final est authoritative cote backend. Le frontend collecte, affiche et pre-valide, mais ne decide jamais seul.

3. Frontends

Customer Storefront

- Next.js + TypeScript
- Tailwind CSS + composants maison/headless
- Design Pack Storefront NOMA
- CSS variables / design tokens
- Approche mobile-first
- Responsive Flexbox / CSS Grid
- RSC / SSR, streaming, Suspense
- Core Web Vitals prioritaires
- Budget JavaScript maitrise et justifie par la performance/UX
- Images AVIF/WebP, cache et prefetch cible

Admin Backoffice

- Next.js + TypeScript
- Tailwind CSS + shadcn/ui + Radix UI
- Design Pack Admin NOMA coherent avec le Storefront
- Desktop-first responsive
- RBAC et permissions fines
- Meme APIs metier que le Storefront
- Aucun acces direct aux bases de donnees

4. Backend microservices

Le backend est standardise en Go. Les principaux domaines sont :

- Catalog
- Product
- Pricing
- Inventory
- Cart
- Order
- Payment
- Tax
- Shipping
- Tracking
- Returns
- Billing
- Fraud/Risk
- Search
- Review
- User Profile
- Notification

Flux	Technologie
Frontend / externe	REST/JSON
Inter-services synchrones	gRPC + Protobuf
Evenements metier durables	Kafka
Jobs / taches operationnelles	RabbitMQ
Source de verite par domaine	PostgreSQL

5. Architecture evenementielle

Kafka et RabbitMQ sont complementaires : Kafka porte les faits metier durables et RabbitMQ les jobs operationnels.

- Outbox Pattern
- Idempotence obligatoire
- Retries et DLQ

- Schema versioning
- Correlation ID, causation ID et trace ID
- aggregate_id comme cle de partition
- aggregate_version
- Apicurio Registry + Protobuf

6. CQRS, lecture et ecriture

Les chemins de lecture et d ecriture sont separes lorsque cela apporte un benefice reel. PostgreSQL reste authoritative pour les decisions transactionnelles.

Usage	Source de lecture
Decision transactionnelle	PostgreSQL du service proprietaire
Recherche / catalogue / facettes	OpenSearch et Redis selon le cas
Analytics / BI	ClickHouse

OpenSearch, Redis et ClickHouse sont des projections ou caches reconstruisibles, jamais des sources de verite metier.

7. Search et discovery

Search est un microservice dedie, alimente par Kafka a partir de Product, Catalog, Pricing, Inventory et Review.

- Full-text
- Autocomplete
- Typo tolerance
- Facettes et filtres
- Ranking et tri
- Synonymes et suggestions
- Reindexation sans downtime
- Search analytics

Pelias + libpostal restent separes et sont utilises pour l autocomplete, la normalisation et le geocodage des adresses.

8. IAM, autorisation et secrets

- Keycloak : OIDC/OAuth2, MFA, Passkeys, access/refresh tokens, rotation et revocation
- Kong : validation OIDC/JWT en entree
- Istio Ambient + SPIFFE/SPIRE : identite workload et mTLS
- OPA : Policy Decision Point de la Platform API
- Kyverno : admission et conformite Kubernetes
- OpenBao : secrets, credentials dynamiques, PKI et TTL courts

Les roles restent des agregats de permissions fines comme product:write, inventory:adjust ou order:refund.

9. Paiement et securite financiere

La plateforme ne stocke jamais de PAN complet ni de CVV. Le paiement utilise tokenisation / Hosted Fields ou mecanisme equivalent.

- Hyperswitch pour l orchestration paiement
- PSP / banques / acquereurs restent externes
- Webhooks via Kong avec signature, anti-replay, idempotence, schema validation et rate limiting
- Validation backend obligatoire avant confirmation frontend
- Reconciliation periodique avec la source externe authoritative

10. Integrations OSS externes

Besoin	Solution OSS retenue
Orchestration paiement	Hyperswitch
SMS	Jasmin + SMPP
Email transactionnel	Postal
Notification multicanal	Novu
Adresse / geocodage	Pelias + libpostal
Transporteurs	Shipping Service + carrier adapters directs

11. Kubernetes et Platform Engineering

La cible est deux sites actifs, chacun avec son propre cluster Kubernetes autonome.

- Cilium + Hubble
- Istio Ambient
- FluxCD
- OpenBao
- CloudNativePG
- Strimzi Kafka
- RabbitMQ Operator
- Redis Cluster
- OpenSearch
- Rook-Ceph + LocalPV NVMe
- Prometheus/Grafana/Loki/Tempo

Rancher est la couche officielle de management multi-cluster. Terraform provisionne l infrastructure. Ansible + AWX assurent hardening, bootstrap et maintenance OS. Cluster API est utilise lorsque le provider est suffisamment mature.

12. CI/CD et GitOps

Git reste la source de verite.

- Tekton pour la CI
- Harbor comme registry OCI

- Trivy pour les scans
- Syft pour SBOM
- Cosign pour signatures
- FluxCD pour le CD/GitOps
- Flagger pour progressive delivery
- Images de production referencees par digest SHA256

Aucun outil ou IA ne doit creer un chemin parallele de deploiement direct vers la production.

13. Platform API et Backstage

Backstage est l UX principale de l Internal Developer Platform. La Platform API constitue le contrat de plateforme et la CLI sert aux developpeurs/automatisations.

- API intent-based
- Idempotence obligatoire
- Operations longues asynchrones
- Versioning N/N-1
- Contract testing
- Audit request_id / change_id / actor / PR / resultat
- OPA : ALLOW / APPROVAL / DENY
- Toute operation produit du declaratif / PR GitOps

14. Observabilite, SRE et AIOps

- Prometheus + Alertmanager
- Thanos pour la vue multi-site et le long terme
- Grafana
- Loki
- Tempo
- OpenTelemetry
- Hubble
- Splunk pour les evenements securite cibles
- Falco
- K8sGPT + KubeBuddy pour diagnostic en lecture seule

L IA peut diagnostiquer et proposer une correction, mais aucune remediation autonome directe en production n est autorisee. Le chemin reste diagnostic -> proposition -> PR -> CI/policies -> FluxCD.

15. Data Platform

Le pipeline Data est construit autour de Debezium + Kafka Connect + ClickHouse + dbt Core + DataHub + Superset.

- CDC PostgreSQL -> Kafka -> ClickHouse pour les usages Data cibles
- Les evenements metier Outbox restent la reference inter-services
- DataHub : catalogue / lineage / ownership data

- Superset : BI
- dbt : transformations et Data Quality-as-Code

16. Stockage

Type	Usage
LocalPV NVMe	PostgreSQL critique, Kafka, workloads IOPS
Ceph RBD	Volumes bloc RWO generalistes
CephFS	RWX uniquement si necessaire
MinIO / objet OSS	Harbor blobs, Loki, Tempo, backups et objets

Chaque site possede son propre cluster Ceph. Aucun Ceph etendu sur le WAN. La replication n est jamais consideree comme un backup.

17. Load balancing et reseau

- PowerDNS + dnsmist : GSLB multi-site
- HAProxy : load balancer d entree API par site
- Apache Traffic Server : web/CDN et origins
- Kubernetes Services + Cilium : load balancing interne
- Istio Ambient : traffic policies et canary
- PgBouncer + CloudNativePG : writer / read replicas
- Redis Cluster : slots natifs
- Kafka : decouverte native des brokers
- RabbitMQ : Kubernetes Service + Quorum Queues
- Squid : proxy egress OSS derriere Istio Egress Gateway

18. Haute disponibilite et multi-site

Le failover multi-site suit une strategie d ownership des ecritures avec home_site. Un seul site possede le droit d ecrire un agregat a un instant donne.

- Health checks fiables
- Quorum et fencing
- Promotion controlee
- Prevention stricte du split-brain
- Kafka par site + MirrorMaker 2
- DNS failover separe du mecanisme de fencing
- Failback-as-Code
- Data reconciliation apres failover/failback

19. Objectifs de volumetrie et performance

Indicateur	Cible
Comptes	500 000

Utilisateurs actifs / jour	50 000
Utilisateurs simultanés au pic	10 000
Commandes nominal / pic	20 000 / 50 000 par jour
REST nominal / pic	500 / 2 000 RPS
gRPC nominal / pic	1 000 / 5 000 req/s
Kafka nominal / pic / burst	1 000 / 5 000 / 10 000 events/s
API publique SLO	99,9 %
Order / Payment SLO	99,95 %
REST p95 / p99	< 300 ms / < 1 s
gRPC p95 / p99	< 100 ms / < 300 ms
Search p95	< 250 ms
Kafka consumer lag normal	< 30 s

20. Quality Engineering

La qualité est automatisée et intégrée aux quality gates CI/CD.

- TDD pour les invariants critiques
- Go testing + testcontainers-go
- OpenAPI + Schemathesis
- Buf + Apicurio pour gRPC/Kafka
- Vitest + Testing Library
- Playwright E2E + visual regression
- axe-core
- Lighthouse CI
- k6
- Go fuzzing
- Chaos Mesh
- Semgrep + gosec
- Trivy + Gitleaks + OWASP ZAP
- Tests restore/PRA

21. Sécurité et Privacy

- Privacy-by-design
- Data minimization
- Least privilege
- Zero Trust
- Retention-as-Code
- Audit
- Encryption-at-rest et in-transit
- SBOM et signatures
- Policy-as-Code
- Kubernetes encryption-at-rest + KMS OpenBao/HSM
- Git Governance + branches protégées + CODEOWNERS + double approbation Tier 0

- Backup 3-2-1-1-0 avec immutabilite et restore tests

22. PRA et Production Readiness

Le passage en production doit etre base sur des preuves, pas sur une simple impression de stabilite.

- PRA-as-Code
- Reconstruction complete d un site depuis zero
- Failover et failback testes
- RTO/RPO mesures reellement
- Backup restore teste
- Chaos engineering
- Security gates
- Capacity validation
- Data reconciliation
- Runbooks et ownership
- Decision finale GO / CONDITIONAL GO / NO-GO

23. Strategie economique

La phase initiale est OSS/self-hosted first afin d exploiter au mieux les equipes internes deja financees. Les services manages ne sont pas privileges uniquement pour reduire la charge operationnelle.

L architecture reste cependant managed-ready grace a l usage de standards portables : S3, PostgreSQL, Kafka, Redis, OpenTelemetry, OCI, OIDC, OpenAPI et gRPC.

24. Les 16 prompts IA de developpement

#	Prompt
01	Monorepo Backend Go
02	Golden Service Product
03	Product -> Kafka -> Search -> OpenSearch -> Next.js
04	Catalog + Pricing + Inventory + Cart
05	Checkout + Order Saga + Tax + Fraud + Payment
06	Shipping + Tracking + Billing + Notification + Returns
07	Data Platform / CDC / ClickHouse / dbt / DataHub
08	Platform / Kubernetes / GitOps / Observability / PRA
09	IAM / Keycloak / RBAC / MFA / Passkeys
10	Customer Storefront
11	Admin Backoffice
12	Search / Discovery
13	SRE / AIOps
14	Quality Engineering
15	Security / Privacy / Compliance
16	Production Readiness / PRA / GO-NO-GO

Design Pack Storefront NOMA : termine. Design Pack Admin NOMA : termine.

25. Objectif final

Livrer une plateforme e-commerce industrielle, open source, cloud-native, multi-site, event-driven, securisee, observable, testable et reproductible, suffisamment automatisee pour etre operee efficacement par les equipes internes et suffisamment portable pour migrer ulterieurement vers des services manages sans reecriture complete.

La phase de conception macro et de specification des grands prompts est pratiquement terminee. La prochaine etape logique est l execution des Prompts 01 a 16 : developpement, tests, integration des vertical slices, puis certification production.